Assignent 11:

```
/**
 *
================================================================================
 * KBTCOE Connect — DOM Manipulation Engine (app.js)
 * Assignment 11: DOM Manipulation — Live Like Counter & Comment Thread
 *
 * Demonstrates:
 * 1. Element retrieval: getElementById, querySelector, querySelectorAll
 * 2. Event listeners & Event Delegation (single listener on comments container)
 * 3. Dynamic node creation (createElement) and insertion (prepend, appendChild)
 * 4. Dynamic node removal (element.remove())
 * 5. Safe text rendering via textContent (preventing XSS injections)
 * 6. Class list manipulation (classList.add, classList.remove,
classList.toggle)
 * 7. Attribute updates (setAttribute, aria-pressed, aria-label, aria-invalid)
 * 8. Form handling, preventDefault(), whitespace trimming, and validation
 * 9. Synchronized live counters (Likes, Comments) and conditional empty states
 * 10. Accessible status announcements via role="status" and aria-live="polite"
 *
================================================================================
 */

"use strict";

//
================================================================================
// 1. APPLICATION STATE & DATA CONFIGURATION
//
================================================================================
const appState = {
  // Post Like State
  initialLikeCount: 24,
  currentLikeCount: 24,
  isLiked: false,

  // Fictional Active User Profile
  currentUser: {
    id: "user-ts-2026",
    name: "Tanmay Shinde",
    branch: "BE • Comp",
    role: "Current Student (You)",
    initials: "TS"
  }
};

//
================================================================================
// 2. PURE HELPER & LOGIC FUNCTIONS (Exportable for Unit Testing)
//
================================================================================
```

```
/**
 * Validates a comment string to ensure it is not empty or whitespace only.
 * @param {string} text - Raw input string
 * @returns {{ isValid: boolean, trimmedText: string, error: string }}
 */
function validateCommentInput(text) {
  if (typeof text !== "string") {
    return {
      isValid: false,
      trimmedText: "",
      error: "Invalid input type. Comment must be text."
    };
  }

  const trimmed = text.trim();

  if (trimmed.length === 0) {
    return {
      isValid: false,
      trimmedText: "",
      error: "Please enter a comment before submitting. Empty or whitespace-only
submissions are not allowed."
    };
  }

  if (trimmed.length > 400) {
    return {
      isValid: false,
      trimmedText: trimmed,
      error: "Comment exceeds the maximum allowed length of 400 characters."
    };
  }

  return {
    isValid: true,
    trimmedText: trimmed,
    error: ""
  };
}

/**
 * Calculates new like count and active boolean state upon toggle.
 * Enforces non-negative count constraints.
 *
 * @param {boolean} currentLiked - Whether the user currently likes the post
 * @param {number} currentCount - Current displayed like count
 * @param {number} initialCount - Baseline post like count
 * @returns {{ isLiked: boolean, newCount: number }}
 */
function calculateLikeToggle(currentLiked, currentCount, initialCount) {
  const isLiked = !currentLiked;
  let newCount = isLiked ? (initialCount + 1) : initialCount;
```

```
  // Enforce non-negative constraint
  newCount = Math.max(0, newCount);

  return {
    isLiked,
    newCount
  };
}

/**
 * Creates a structured data object for a new comment record.
 * @param {string} text - Sanitized comment text
 * @param {Object} user - Author metadata
 * @returns {Object}
 */
function createCommentData(text, user) {
  return {
    id: "comment-" + Date.now() + "-" + Math.floor(Math.random() * 1000),
    text: text,
    authorName: user.name,
    authorBranch: user.branch,
    authorInitials: user.initials,
    createdAt: new Date().toISOString(),
    displayTime: "Just now"
  };
}

//
============================================================================
// 3. SAFE DOM NODE FACTORY
//
============================================================================

/**
 * Safely constructs a dynamic comment card element using DOM APIs.
 * Strictly uses textContent for all user-generated strings to prevent XSS.
 *
 * @param {Object} comment - Comment data object
 * @returns {HTMLElement} article element representing the comment card
 */
function createCommentCardElement(comment) {
  // 1. Root Card Container: <article class="comment-card newly-added"
data-comment-id="...">
  const card = document.createElement("article");
  card.className = "comment-card newly-added";
  card.dataset.commentId = comment.id;

  // 2. Header Container: <div class="comment-header">
  const header = document.createElement("div");
  header.className = "comment-header";

  // Avatar: <div class="comment-avatar" aria-hidden="true">TS</div>
  const avatar = document.createElement("div");
  avatar.className = "comment-avatar";
```

```
  avatar.setAttribute("aria-hidden", "true");
  avatar.textContent = comment.authorInitials || "ST";

  // Meta details: <div class="comment-meta">
  const meta = document.createElement("div");
  meta.className = "comment-meta";

  const authorRow = document.createElement("div");
  authorRow.className = "comment-author-row";

  const authorName = document.createElement("h4");
  authorName.className = "comment-author";
  authorName.textContent = comment.authorName;

  const branchTag = document.createElement("span");
  branchTag.className = "comment-branch-tag";
  branchTag.textContent = comment.authorBranch;

  authorRow.appendChild(authorName);
  authorRow.appendChild(branchTag);

  const timeEl = document.createElement("time");
  timeEl.className = "comment-time";
  timeEl.setAttribute("datetime", comment.createdAt);
  timeEl.textContent = comment.displayTime;

  meta.appendChild(authorRow);
  meta.appendChild(timeEl);

  // Delete Button: <button type="button" class="btn-delete-comment"
data-action="delete" ...>
  const deleteBtn = document.createElement("button");
  deleteBtn.type = "button";
  deleteBtn.className = "btn-delete-comment";
  deleteBtn.dataset.action = "delete";
  deleteBtn.setAttribute("aria-label", `Delete comment by
${comment.authorName}`);
  deleteBtn.setAttribute("title", "Delete this comment");
  deleteBtn.textContent = "🗑 Delete";

  header.appendChild(avatar);
  header.appendChild(meta);
  header.appendChild(deleteBtn);

  // 3. Comment Body: <div class="comment-body"><p
class="comment-text">...</p></div>
  const body = document.createElement("div");
  body.className = "comment-body";

  const textP = document.createElement("p");
  textP.className = "comment-text";

  // CRITICAL SECURITY REQUIREMENT:
  // Use textContent to safely escape any HTML, preventing script execution.
```

```
  textP.textContent = comment.text;

  body.appendChild(textP);

  // 4. Assemble Card
  card.appendChild(header);
  card.appendChild(body);

  return card;
}

//
================================================================================
// 4. UI SYNCHRONIZATION & STATUS ANNOUNCEMENTS
//
================================================================================

/**
 * Announces status changes to screen readers and visually flashes feedback.
 * @param {string} message - Message to announce
 * @param {'info' | 'success' | 'danger'} type - Status theme
 */
function announceStatus(message, type = "info") {
  const statusEl = document.getElementById("actionStatus");
  if (!statusEl) return;

  statusEl.textContent = message;
  statusEl.className = `action-status-banner status-${type}`;
  statusEl.classList.remove("visually-hidden");

  // Automatically fade after 3.5 seconds
  if (statusEl.fadeTimeout) {
    clearTimeout(statusEl.fadeTimeout);
  }

  statusEl.fadeTimeout = setTimeout(() => {
    statusEl.classList.add("visually-hidden");
  }, 3500);
}

/**
 * Synchronizes all comment count indicators across the page and
 * conditionally renders the empty state when comment list is empty.
 *
 * Demonstrates:
 * - querySelector / querySelectorAll
 * - textContent update
 * - classList.add / remove
 */
function syncCommentCounters() {
  const commentsList = document.getElementById("commentsList");
  const emptyState = document.getElementById("emptyCommentsState");
  if (!commentsList) return 0;
```

```
  // Retrieve all comment cards currently in the DOM
  const commentCards = commentsList.querySelectorAll(".comment-card");
  const count = commentCards.length;

  // Update stat bar counter
  const statCommentCount = document.getElementById("statCommentCount");
  if (statCommentCount) {
    statCommentCount.textContent = String(count);
  }

  // Update toolbar badge counter
  const commentCountBadge = document.getElementById("commentCountBadge");
  if (commentCountBadge) {
    commentCountBadge.textContent = String(count);
  }

  // Update section header pill
  const headerCommentCount = document.getElementById("headerCommentCount");
  if (headerCommentCount) {
    headerCommentCount.textContent = String(count);
  }

  // Conditional Rendering of Empty State Card
  if (emptyState) {
    if (count === 0) {
      emptyState.classList.remove("visually-hidden");
    } else {
      emptyState.classList.add("visually-hidden");
    }
  }

  return count;
}

/**
 * Updates the Like Button UI, icon, text, and counter attributes.
 * Demonstrates:
 * - classList.toggle
 * - setAttribute
 * - textContent
 */
function syncLikeButtonUI() {
  const btnLike = document.getElementById("btnLike");
  const badgeCount = document.getElementById("likeCountBadge");
  const statLikeCount = document.getElementById("statLikeCount");
  if (!btnLike || !badgeCount || !statLikeCount) return;

  const { isLiked, currentLikeCount } = appState;

  // 1. Update text counters
  badgeCount.textContent = String(currentLikeCount);
  statLikeCount.textContent = String(currentLikeCount);

  // 2. Toggle active liked class
```

```
  btnLike.classList.toggle("liked", isLiked);

  // 3. Update accessibility attributes
  btnLike.setAttribute("aria-pressed", String(isLiked));
  btnLike.setAttribute(
    "aria-label",
    isLiked
      ? `Unlike this post. Current count: ${currentLikeCount}`
      : `Like this post. Current count: ${currentLikeCount}`
  );

  // 4. Update heart icon and label inside button
  const iconEl = btnLike.querySelector(".heart-icon");
  const textEl = btnLike.querySelector(".btn-like-text");

  if (iconEl) {
    iconEl.textContent = isLiked ? "♡" : "♡";
  }
  if (textEl) {
    textEl.textContent = isLiked ? "Liked" : "Like";
  }
}

//
============================================================================
// 5. EVENT HANDLERS
//
============================================================================

/**
 * Toggles the Like status of the post.
 */
function handleLikeClick() {
  const result = calculateLikeToggle(
    appState.isLiked,
    appState.currentLikeCount,
    appState.initialLikeCount
  );

  appState.isLiked = result.isLiked;
  appState.currentLikeCount = result.newCount;

  syncLikeButtonUI();

  if (appState.isLiked) {
    announceStatus(`You liked this post. Total likes:
${appState.currentLikeCount}`, "success");
  } else {
    announceStatus(`You unliked this post. Total likes:
${appState.currentLikeCount}`, "info");
  }
}

/**
```

```
 * Handles live keystroke input in the comment textarea.
 * Dynamically updates character counter and clears validation error state.
 */
function handleCommentInput(e) {
  const textarea = e.target;
  const charCounter = document.getElementById("commentCharCount");
  const errorBanner = document.getElementById("commentError");

  const length = textarea.value.length;
  if (charCounter) {
    charCounter.textContent = `${length} / 400`;
  }

  // Clear validation error when user begins typing
  if (errorBanner && !errorBanner.classList.contains("visually-hidden")) {
    errorBanner.classList.add("visually-hidden");
    textarea.classList.remove("is-invalid");
    textarea.removeAttribute("aria-invalid");
  }
}

/**
 * Handles new comment form submission.
 * Validates input, constructs new DOM card, prepends to thread, and updates
counters.
 */
function handleCommentSubmit(e) {
  e.preventDefault();

  const inputEl = document.getElementById("commentInput");
  const errorBanner = document.getElementById("commentError");
  const commentsList = document.getElementById("commentsList");
  if (!inputEl || !commentsList) return;

  const validation = validateCommentInput(inputEl.value);

  // 1. Validation check
  if (!validation.isValid) {
    if (errorBanner) {
      errorBanner.textContent = `⚠ ${validation.error}`;
      errorBanner.classList.remove("visually-hidden");
    }
    inputEl.classList.add("is-invalid");
    inputEl.setAttribute("aria-invalid", "true");
    inputEl.focus();
    announceStatus(`Validation Error: ${validation.error}`, "danger");
    return;
  }

  // 2. Clear error state
  if (errorBanner) {
    errorBanner.classList.add("visually-hidden");
  }
  inputEl.classList.remove("is-invalid");
```

```
  inputEl.removeAttribute("aria-invalid");

  // 3. Construct data and DOM card
  const commentData = createCommentData(validation.trimmedText,
appState.currentUser);
  const cardElement = createCommentCardElement(commentData);

  // 4. Prepend to comments list (newest comment appears first)
  commentsList.prepend(cardElement);

  // 5. Reset input form & char count
  inputEl.value = "";
  const charCounter = document.getElementById("commentCharCount");
  if (charCounter) {
    charCounter.textContent = "0 / 400";
  }

  // 6. Synchronize counters & empty state
  const totalComments = syncCommentCounters();

  // 7. Announce success
  announceStatus("Your comment was posted successfully!", "success");

  // Keep focus on input for quick subsequent typing
  inputEl.focus();
}

/**
 * Event Delegation Handler on Comments List Container.
 * Captures clicks from any dynamically added Delete button without attaching
 * separate listeners to every single comment node.
 *
 * Demonstrates:
 * - e.target
 * - closest()
 * - node.remove()
 */
function handleCommentsListClick(e) {
  // Check if click originated from or inside a delete button
  const deleteBtn = e.target.closest(".btn-delete-comment");
  if (!deleteBtn) return;

  // Identify target comment card
  const commentCard = deleteBtn.closest(".comment-card");
  if (!commentCard) return;

  const authorName = commentCard.querySelector(".comment-author")?.textContent
|| "User";

  // Prevent multiple rapid clicks from throwing errors
  deleteBtn.disabled = true;

  // Remove comment node from DOM
  commentCard.remove();
```

```
  // Synchronize counters
  const remaining = syncCommentCounters();

  // Announce status
  announceStatus(`Deleted comment by ${authorName}. Remaining comments:
${remaining}`, "info");
}

/**
 * Handles clearing all comments with user confirmation.
 * Demonstrates:
 * - window.confirm
 * - querySelectorAll
 * - remove()
 */
function handleClearAllComments() {
  const commentsList = document.getElementById("commentsList");
  if (!commentsList) return;

  const commentCards = commentsList.querySelectorAll(".comment-card");
  if (commentCards.length === 0) {
    announceStatus("The comment thread is already empty.", "info");
    return;
  }

  const confirmed = window.confirm(
    `Are you sure you want to remove all ${commentCards.length} comments from
this post? This action cannot be undone.`
  );

  if (!confirmed) {
    announceStatus("Clear comments cancelled. Comments were preserved.",
"info");
    return;
  }

  // Remove all comment elements
  commentCards.forEach(card => card.remove());

  // Synchronize counters and restore empty state
  syncCommentCounters();

  announceStatus("All comments have been cleared.", "danger");
}

/**
 * Auxiliary Share button handler demonstrating class toggle and feedback.
 */
function handleShareClick() {
  const currentUrl = window.location.href;
  if (navigator.clipboard && navigator.clipboard.writeText) {
    navigator.clipboard.writeText(currentUrl).then(() => {
      announceStatus("Post link copied to clipboard!", "success");
```

```
    }).catch(() => {
      announceStatus("Post link: " + currentUrl, "info");
    });
  } else {
    announceStatus("Post link: " + currentUrl, "info");
  }
}

//
================================================================================
// 6. DOM READY INITIALIZATION
//
================================================================================
function initDOMManipulationApp() {
  console.log("Initializing KBTCOE Connect — DOM Manipulation Practical
(Assignment 11)...");

  // 1. Like Button
  const btnLike = document.getElementById("btnLike");
  if (btnLike) {
    btnLike.addEventListener("click", handleLikeClick);
  }

  // 2. Comment Form & Input
  const commentForm = document.getElementById("commentForm");
  const commentInput = document.getElementById("commentInput");

  if (commentForm) {
    commentForm.addEventListener("submit", handleCommentSubmit);
  }
  if (commentInput) {
    commentInput.addEventListener("input", handleCommentInput);
  }

  // 3. Event Delegation on Comments List Container
  const commentsList = document.getElementById("commentsList");
  if (commentsList) {
    commentsList.addEventListener("click", handleCommentsListClick);
  }

  // 4. Clear All Comments Button
  const btnClearComments = document.getElementById("btnClearComments");
  if (btnClearComments) {
    btnClearComments.addEventListener("click", handleClearAllComments);
  }

  // 5. Share Button
  const btnShare = document.getElementById("btnShare");
  if (btnShare) {
    btnShare.addEventListener("click", handleShareClick);
  }

  // 6. Initial Synchronizations
  syncLikeButtonUI();
```

```
  syncCommentCounters();
}

// Attach listener once document is parsed
if (typeof document !== "undefined") {
  if (document.readyState === "loading") {
    document.addEventListener("DOMContentLoaded", initDOMManipulationApp);
  } else {
    initDOMManipulationApp();
  }
}

//
// ============================================================================
// 7. NODE.JS EXPORTS FOR AUTOMATED TESTING
//
// ============================================================================
if (typeof module !== "undefined" && module.exports) {
  module.exports = {
    appState,
    validateCommentInput,
    calculateLikeToggle,
    createCommentData,
    createCommentCardElement,
    syncCommentCounters,
    syncLikeButtonUI
  };
}
```